

3B2 FPGA Programming Lab Report

Liu Zihe^{1*}

¹University of Cambridge

Abstract

This report documents the design, implementation and testing of a traffic light controller on an Altera Cyclone V FPGA. A three-state finite state machine controlling a pedestrian crossing is described in Verilog, compiled, and configured onto a DE1-SoC board via JTAG, and is improved with a vehicle lockout.

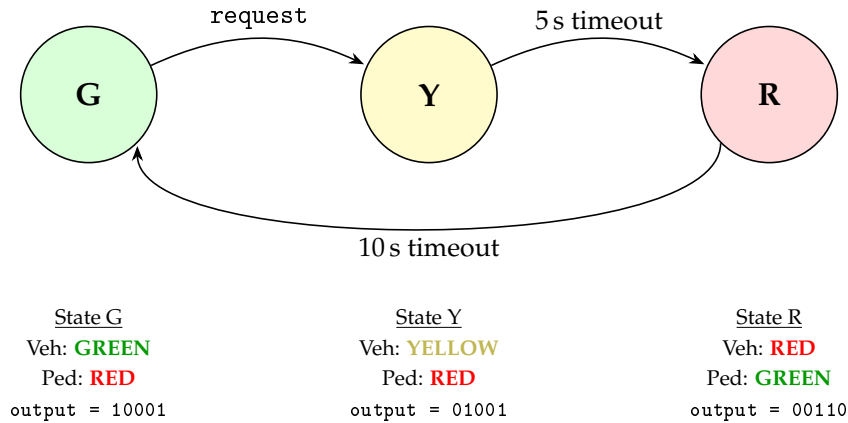


Figure 1: State diagram for the traffic light controller FSM. The 5-bit output vector is {veh_G, veh_Y, veh_R, ped_G, ped_R}. reset (active LOW, asynchronous) returns to state G from any state.

Listing 1: Traffic Light Controller

```
1 // =====
2 // Traffic Light Controller (TLC) for Pedestrian Crossing
3 // =====
4 // Implements a 3-state FSM for a pedestrian light-controlled crossing.
5 //
6 // FSM states:
7 //   State G (initial) - vehicle GREEN,  pedestrian RED
8 //   State Y           - vehicle YELLOW, pedestrian RED  (transitional)
9 //   State R           - vehicle RED,   pedestrian GREEN
10 //
11 // Behaviour:
12 //   - Initial state: State G (vehicle GREEN, pedestrian RED)
13 //   - On pedestrian request: State G -> State Y (5s) -> State R (10s) -> State G
14 //   - Reset returns to State G at any time
15 //
16 // Target: DE1-SoC board (Cyclone V FPGA) with 50 MHz clock
17 // Output mapping: {veh_GREEN, veh_YELLOW, veh_RED, ped_GREEN, ped_RED}
18 // =====
19
20 module tlc (
21     input wire clk, // 50 MHz on-board clock (PIN_AF14)
22     input wire request, // Pedestrian push-button (active LOW on DE1-SoC)
23     input wire reset, // Reset push-button (active LOW, asynchronous)
24     output reg [4:0] \output // 5-bit output: {veh_GREEN, veh_YELLOW, veh_RED, ped_GREEN, ped_RED}
25 );
26
27 // --- State encoding (2-bit) ---
28 // Only 3 states needed, so 2 bits suffice
29 localparam G = 2'd0; // State G: vehicle GREEN / pedestrian RED
30 localparam Y = 2'd1; // State Y: vehicle YELLOW / pedestrian RED
31 localparam R = 2'd2; // State R: vehicle RED / pedestrian GREEN
32
```

*CRSID: zl559. Date: 20th February 2026

```

33 reg [1:0] state; // Current FSM state
34 reg [28:0] count; // 29-bit counter for timing delays
35 // 29 bits can count up to  $2^{29} - 1 = 536,870,911$ 
36 // which is enough for 500,000,000 (10s @ 50 MHz)
37
38 // =====
39 // Sequential logic: state transitions & timer
40 // =====
41 // Triggered on rising clock edge OR falling edge of reset (async reset).
42 // DE1-SoC push-buttons are active LOW, so reset triggers on negedge.
43 always @(posedge clk or negedge reset) begin
44     if (!reset) begin
45         // --- Asynchronous reset: return to State G ---
46         state <= G;
47         count <= 29'd0;
48     end else begin
49         case (state)
50
51             // --- State G: wait for pedestrian request ---
52             // request is active LOW (button pressed = 0), so we check == 0.
53             // No timer; we stay in State G indefinitely until request.
54             G: begin
55                 if (request == 1'b0) begin
56                     state <= Y;
57                     count <= 29'd0;
58                 end
59             end
60
61             // --- State Y: vehicle YELLOW for 5 seconds ---
62             // 5 seconds @ 50 MHz = 250,000,000 clock cycles.
63             Y: begin
64                 if (count == 29'd250_000_000) begin
65                     state <= R;
66                     count <= 29'd0;
67                 end else begin
68                     count <= count + 29'd1;
69                 end
70             end
71
72             // --- State R: vehicle RED, pedestrian GREEN for 10 seconds ---
73             // 10 seconds @ 50 MHz = 500,000,000 clock cycles.
74             R: begin
75                 if (count == 29'd500_000_000) begin
76                     state <= G;
77                     count <= 29'd0;
78                 end else begin
79                     count <= count + 29'd1;
80                 end
81             end
82
83             // --- Default: safety fallback to State G ---
84             // Catches undefined states (e.g. state = 2'd3)
85             default: begin
86                 state <= G;
87                 count <= 29'd0;
88             end
89         endcase
90     end
91 end
92
93 // =====
94 // Combinational logic: output assignment
95 // =====
96 // Maps each state to the 5-bit LED output vector:
97 // Bit [4] = veh_GREEN
98 // Bit [3] = veh_YELLOW
99 // Bit [2] = veh_RED
100 // Bit [1] = ped_GREEN
101 // Bit [0] = ped_RED
102 //
103 // State G: veh_GREEN on, ped_RED on -> 10001
104 // State Y: veh_YELLOW on, ped_RED on -> 01001
105 // State R: veh_RED on, ped_GREEN on -> 00110

```

```
106 | always @(*) begin
107 |     case (state)
108 |         G:         \output = 5'b10001;
109 |         Y:         \output = 5'b01001;
110 |         R:         \output = 5'b00110;
111 |         default: \output = 5'b10001; // Default to State G
112 |     endcase
113 | end
114 |
115 | endmodule
```

References

- [1] T. M. Cover and J. A. Thomas, *Elements of Information Theory*, 2nd ed. Wiley-Interscience, 2006.
- [2] D. J. C. MacKay, *Information Theory, Inference, and Learning Algorithms*. Cambridge University Press, 2003.
- [3] S. Godsill, *Estimation Theory and Inference Methods*, Lecture Notes (3F3), University of Cambridge, 2023
- [4] J. Weisberg, "Laplace's Rule of Succession," *Formal Epistemology*, 2019. Available: <https://jonathanweisberg.org/post/inductive-logic-2/>
- [5] A. Tsun, "The Beta and Dirichlet Distributions," Chapter 7.4 in *Probability & Statistics with Applications to Computing*, CS109 Course Reader, Stanford University, 2025. Available: https://web.stanford.edu/class/archive/cs/cs109/cs109.1218/files/student_drive/7.4.pdf
- [6] E. P. Xing, "Bayesian Nonparametrics: Dirichlet Processes," Lecture 19 in *10-708: Probabilistic Graphical Models*, Lecture notes by C. Malings and J. Gao, Carnegie Mellon University, Spring 2014. Available: https://www.cs.cmu.edu/~epxing/Class/10708-14/scribe_notes/scribe_note_lecture19.pdf